

```

class Demo{
    public static void main(String args[]){
        List list1=new List(100);
        list1.add(10);
        list1.add(20);
        list1.add(30);
        list1.add(40);
        list1.add(50);
        list1.printList(); //[10,20,30,40,50]

        list1.remove(2);
        list1.printList(); //[10,20,40,50]
    }
}

```

Data Structures using “Nodes”

Q97 Step I

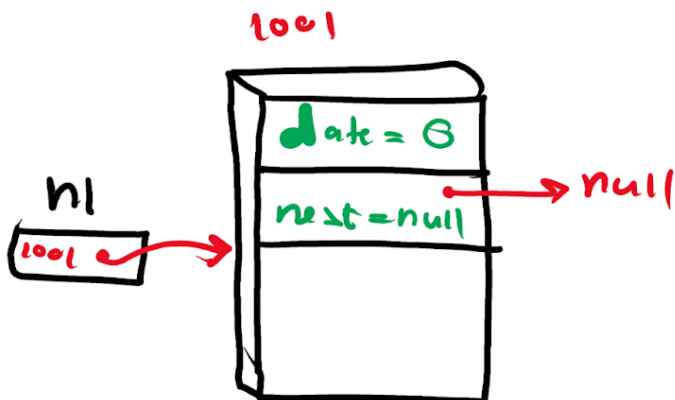
=====

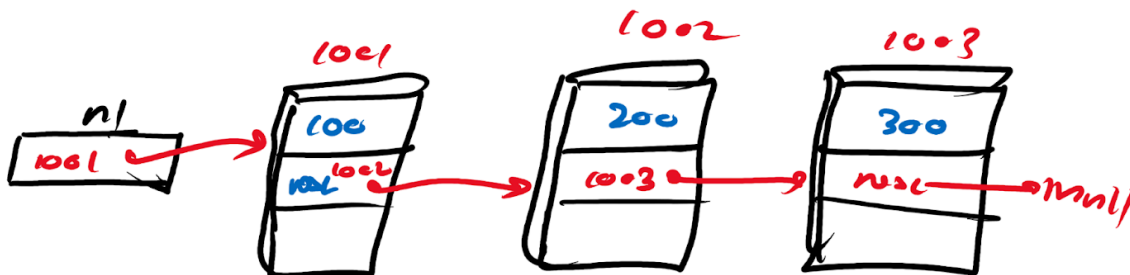
```

class Node{
    int data;
    Node next;
}

class Demo{
    public static void main(String args[]){
        Node n1=new Node();
    }
}

```

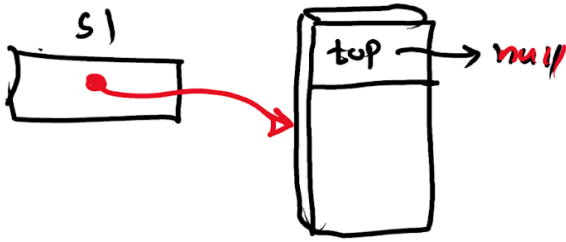




```
class Stack{
    private Node top;
}

class Node{
    int data;
    Node next;
    Node(int data){
        this.data=data;
    }
}

class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
    }
}
```



Q102 Step VI

```
class Stack{
    private Node top;

    public void push(int data){
        //
        //
    }
}

class Node{
    int data;
    Node next;
    Node(int data){
        this.data=data;
    }
}

class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.push(10);
    }
}
```

```

////////////////////////////////////////////////////////////////////////////////////////////////
Q103 From Q102
=====
class Stack{
    private Node top;

    public void push(int data){
        Node n1=new Node(data);
        top=n1;
    }
}

class Node{
    int data;
    Node next;
    Node(int data){
        this.data=data;
    }
}

class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.push(10);
    }
}

```

```

////////////////////////////////////////////////////////////////////////////////////////////////
Q104 Step VII
=====
class Stack{
    private Node top;

    public void push(int data){
        Node n1=new Node(data);
        top=n1;
    }
}

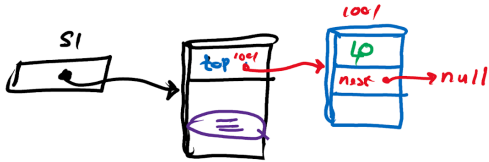
class Node{
    int data;
    Node next;
    Node(int data){
        this.data=data;
    }
}

```

```

class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.push(10);
        s1.push(20);
        s1.push(30);
        s1.push(40);
    }
}

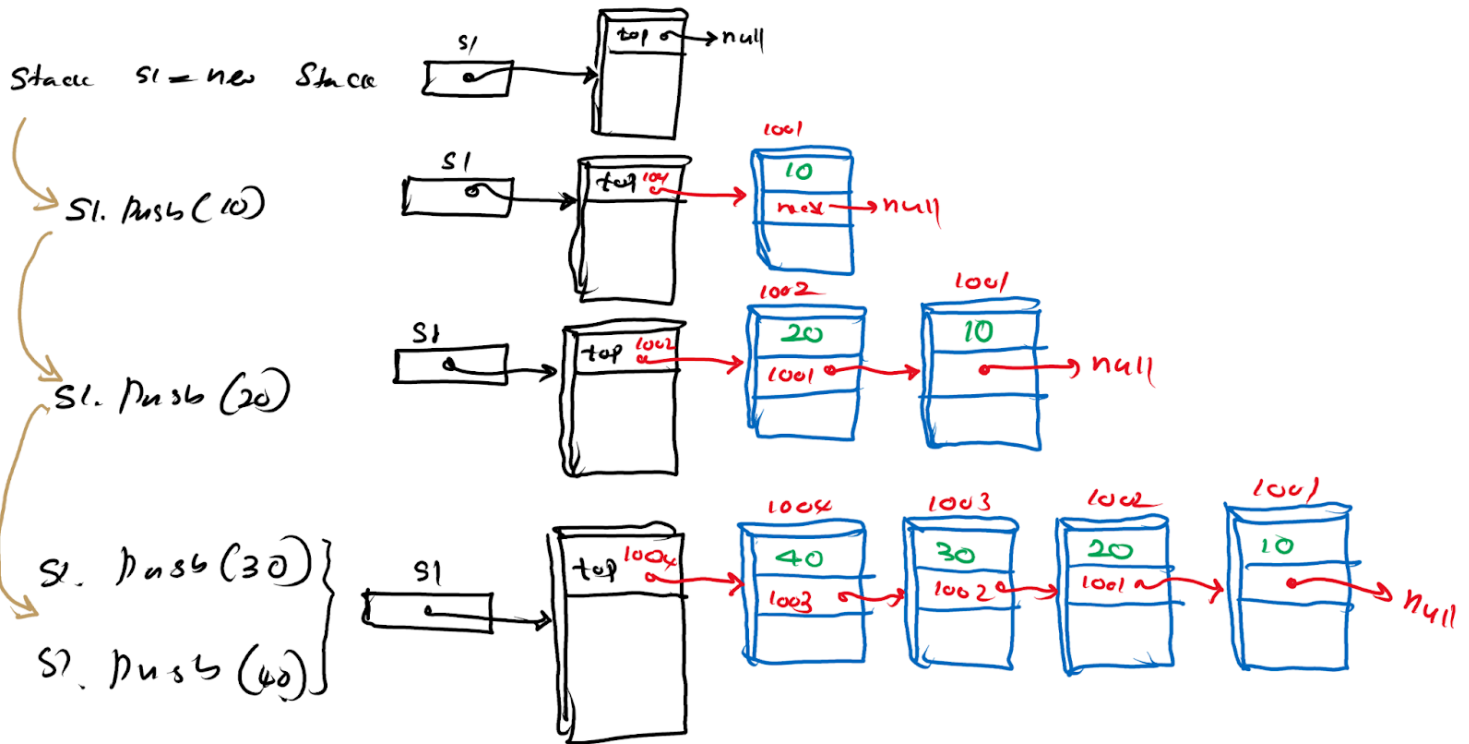
```



////////////////////////////////////

Q105 Step VIII

=====



////////////////////////////////////

Q106 From Q105

=====

```

class Stack{
    private Node top;

    public void push(int data){
        Node n1=new Node(data); //1
        n1.next=top; //2
        top=n1; //3
    }
}

```

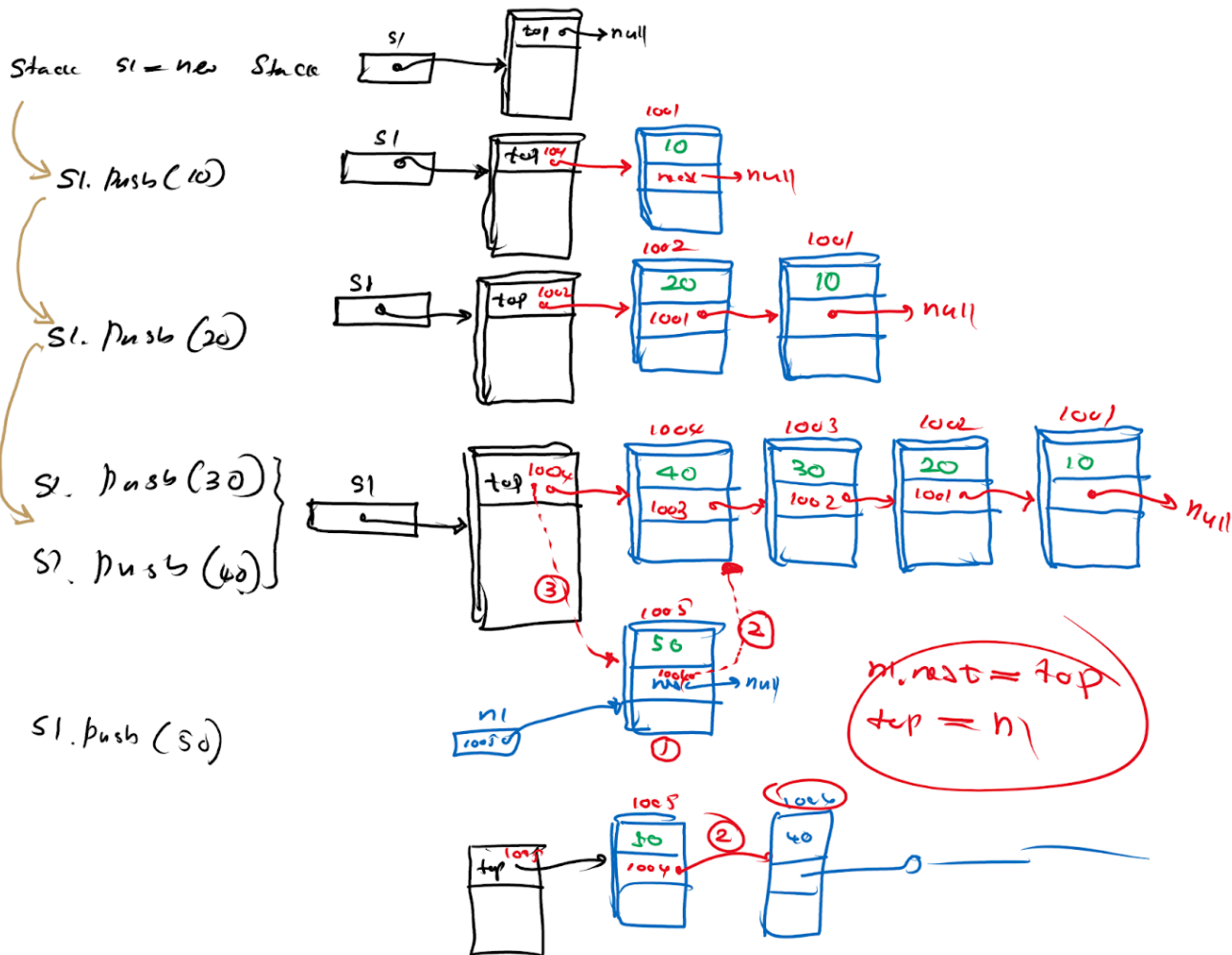
```

class Node{
    int data;
    Node next;
    Node(int data){
        this.data=data;
    }
}

class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.push(10);
        s1.push(20);
        s1.push(30);
        s1.push(40);

        s1.push(50);
    }
}

```



=====

```

class Stack{
    private Node top;

    public void push(int data){
        Node n1=new Node(data);
        n1.next=top;
        top=n1;
    }
    public void printStack(){

    }

}

class Node{
    int data;
    Node next;
    Node(int data){
        this.data=data;
    }

}

class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.push(10);
        s1.push(20);
        s1.push(30);
        s1.push(40);
        s1.push(50);
        s1.printStack(); //[50,40,30,20,10]
    }

}

```

////////////////////////////////////

Q108 From Q107 (Logical error)

=====

```
class Stack{
    private Node top;

    public void push(int data){
        Node n1=new Node(data);
        n1.next=top;
        top=n1;
    }

    public void printStack(){
        System.out.print("[");
        while(top!=null){
```

```

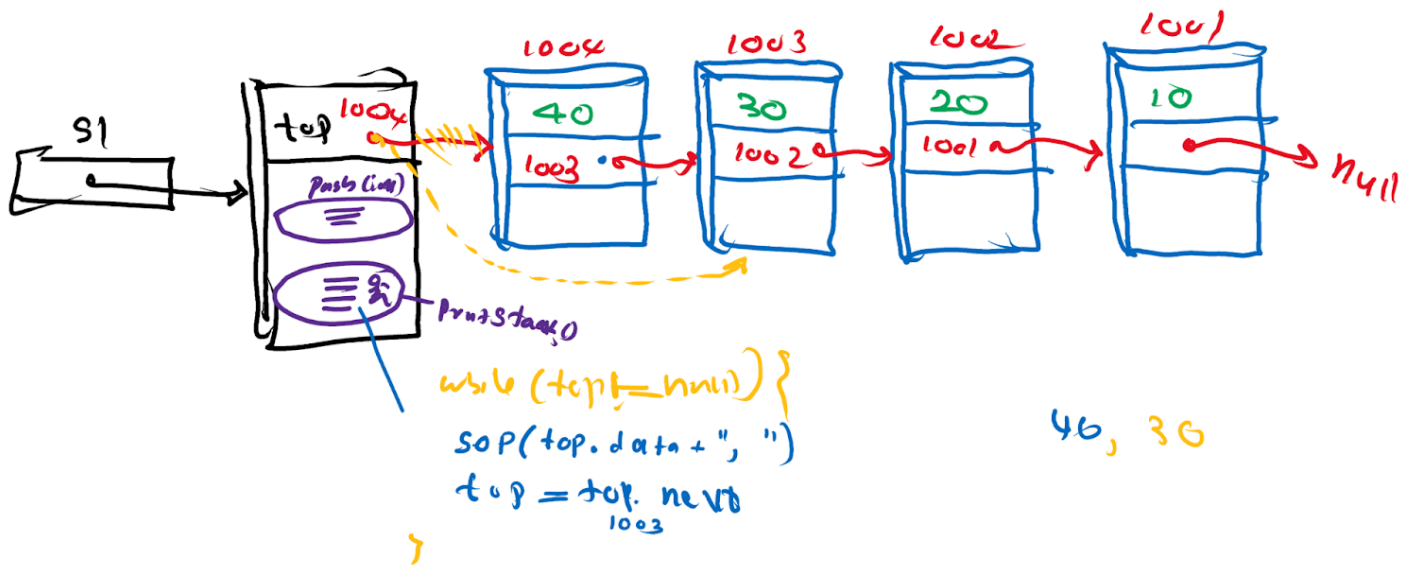
        System.out.print(top.data+" ");
        top=top.next;
    }
    System.out.println("\b\b");
}

class Node{
    int data;
    Node next;
    Node(int data){
        this.data=data;
    }
}

class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.push(10);
        s1.push(20);
        s1.push(30);
        s1.push(40);
        s1.push(50);
        s1.printStack(); //[50,40,30,20,10]
    }
}

```

top = top.next;



=====

```

class Stack{
    private Node top;

    public void push(int data){
        Node n1=new Node(data);
        n1.next=top;
        top=n1;
    }
    public void printStack(){
        Node temp=top;
        System.out.print("[");
        while(temp!=null){
            System.out.print(temp.data+" , ");
            temp=temp.next;
        }
        System.out.println(top==null ? "empty]":"\b\b]");
    }
}

class Node{
    int data;
    Node next;
    Node(int data){
        this.data=data;
    }
}

class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.printStack(); //[empty]
        s1.push(10);
        s1.push(20);
        s1.push(30);
        s1.push(40);
        s1.push(50);
        s1.printStack(); //[50,40,30,20,10]
        s1.printStack(); //[50,40,30,20,10]
    }
}

```

```

class Stack{
    private Node top;

    public void push(int data){
        Node n1=new Node(data);
        n1.next=top;
        top=n1;
    }
    public void printStack(){
        Node temp=top;
        System.out.print("[");
        while(temp!=null){
            System.out.print(temp.data+", ");
            temp=temp.next;
        }
        System.out.println(top==null ? "empty"]":"\b\b]");
    }
}

class Node{
    int data;
    Node next;
    Node(int data){
        this.data=data;
    }
}

class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.printStack(); //[empty]
        s1.push(10);
        s1.push(20);
        s1.push(30);
        s1.push(40);
        s1.push(50);
        s1.printStack(); //[50,40,30,20,10]

        s1.pop();
        s1.printStack(); //[40,30,20,10]

    }
}

```

```

class Stack{
    private Node top;

    public void push(int data){
        Node n1=new Node(data);
        n1.next=top;
        top=n1;
    }
    public void pop(){
        if(top!=null){
            top=top.next;
        }
    }
    public void printStack(){
        Node temp=top;
        System.out.print("[");
        while(temp!=null){
            System.out.print(temp.data+", ");
            temp=temp.next;
        }
        System.out.println(top==null ? "empty"]":"\b\b]");
    }
}

class Node{
    int data;
    Node next;
    Node(int data){
        this.data=data;
    }
}

class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.printStack(); //[empty]
        s1.push(10);
        s1.push(20);
        s1.push(30);
        s1.push(40);
        s1.push(50);
        s1.printStack(); //[50,40,30,20,10]
        s1.pop();
        s1.printStack(); //[40,30,20,10]
    }
}

```

```

class Stack{
    private Node top;

    public void push(int data){
        Node n1=new Node(data);
        n1.next=top;
        top=n1;
    }
    public void pop(){
        if(top!=null){
            top=top.next;
        }
    }
    public void printStack(){
        Node temp=top;
        System.out.print("[");
        while(temp!=null){
            System.out.print(temp.data+" ");
            temp=temp.next;
        }
        System.out.println(top==null ? "empty":"\b\b]");
    }
}

class Node{
    int data;
    Node next;
    Node(int data){
        this.data=data;
    }
}

class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.printStack(); //[empty]
        System.out.println("Size of the stack : "+s1.size()); //0
        s1.push(10);
        s1.push(20);
        s1.push(30);
        s1.push(40);
        s1.push(50);
        s1.printStack(); //[50,40,30,20,10]
        System.out.println("Size of the stack : "+s1.size()); //5
    }
}

```

[illegible]

=====

$$\}$$


```

private boolean isEmpty(){
    return top==null;
}
public void clear(){
    top=null;
}
public void printStack(){
    Node temp=top;
    System.out.print("[");
    while(temp!=null){
        System.out.print(temp.data+", ");
        temp=temp.next;
    }
    System.out.println(top==null ? "empty]":"\b\b]");
}
}
class Node{
    int data;
    Node next;
    Node(int data){
        this.data=data;
    }
}
class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.push(10);
        s1.push(20);
        s1.push(30);
        s1.push(40);
        s1.push(50);
        s1.printStack(); //[50,40,30,20,10]
        System.out.println("Contains 30 : "+s1.contains(30)); //true
        System.out.println("Contains 33 : "+s1.contains(33)); //false

        System.out.println("Index of 30 : "+s1.search(30)); //2
        System.out.println("Index of 33 : "+s1.search(33)); //-1
    }
}

```

```
class Stack{
    private Node top;

    public void push(int data){
        Node n1=new Node(data);
        n1.next=top;
        top=n1;
    }
    public void pop(){
        if(top!=null){
            top=top.next;
        }
    }
    public int search(int data){
        Node temp=top;
        int index=0;
        while(temp!=null){
            if(temp.data==data){
                return index;
            }
            index++;
            temp=temp.next;
        }
        return -1;
    }
    public boolean contains(int data){
        return search(data)!=-1;
    }
    public int size(){
        int count=0;
        Node temp=top;
        while(temp!=null){
            count++;
            temp=temp.next;
        }
        return count;
    }
    private boolean isEmpty(){
        return top==null;
    }
    public void clear(){
        top=null;
    }
}
```

```

        public void printStack(){
            Node temp=top;
            System.out.print("[");
            while(temp!=null){
                System.out.print(temp.data+", ");
                temp=temp.next;
            }
            System.out.println(top==null ? "empty]":"\b\b]");
        }
    }

    class Node{
        int data;
        Node next;
        Node(int data){
            this.data=data;
        }
    }

    class Demo{
        public static void main(String args[]){
            Stack s1=new Stack();
            s1.push(10);
            s1.push(20);
            s1.push(30);
            s1.push(40);
            s1.push(50);
            s1.printStack(); //[50,40,30,20,10]
            System.out.println("Contains 30 : "+s1.contains(30)); //true
            System.out.println("Contains 33 : "+s1.contains(33)); //false

            System.out.println("Index of 30 : "+s1.search(30)); //2
            System.out.println("Index of 33 : "+s1.search(33)); //-1
        }
    }
}

```

```

class Stack{
    private Node top;

    public void push(int data){
        Node n1=new Node(data);
        n1.next=top;
        top=n1;
    }
    public void pop(){
        if(top!=null){
            top=top.next;
        }
    }
    public int search(int data){
        Node temp=top;
        int index=0;
        while(temp!=null){
            if(temp.data==data){
                return index;
            }
            index++;
            temp=temp.next;
        }
        return -1;
    }
    public boolean contains(int data){
        return search(data)!=-1;
    }
    public int size(){
        int count=0;
        Node temp=top;
        while(temp!=null){
            count++;
            temp=temp.next;
        }
        return count;
    }
    private boolean isEmpty(){
        return top==null;
    }
    public void clear(){
        top=null;
    }
    public void printStack(){
        Node temp=top;

```

```

        System.out.print("[");
        while(temp!=null){
            System.out.print(temp.data+", ");
            temp=temp.next;
        }
        System.out.println(top==null ? "empty]":"\b\b]");
    }
}

class Node{
    int data;
    Node next;
    Node(int data){
        this.data=data;
    }
}

class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.push(10);
        s1.push(20);
        s1.push(30);
        s1.push(40);
        s1.push(50);

        s1.printStack(); //[50,40,30,20,10]
        int topElement=s1.peek();
        System.out.println("Top Element : "+topElement);//50

        System.out.println();
        System.out.println("After calling peek()...");
        s1.printStack();
        s1.printStack(); //[50,40,30,20,10]
        System.out.println();

        topElement=s1.poll();
        System.out.println("Top Element : "+topElement);//50

        System.out.println();
        System.out.println("After calling poll()...");
        s1.printStack();
        s1.printStack(); //[40,30,20,10]

    }
}

```

```

class Stack{
    private Node top;

    public void push(int data){
        Node n1=new Node(data);
        n1.next=top;
        top=n1;
    }
    public void pop(){
        if(top!=null){
            top=top.next;
        }
    }
    public int search(int data){
        Node temp=top;
        int index=0;
        while(temp!=null){
            if(temp.data==data){
                return index;
            }
            index++;
            temp=temp.next;
        }
        return -1;
    }
    public int peek(){
        return isEmpty()? -1:top.data;
    }
    public int poll(){
        int topElement=isEmpty()? -1:top.data;
        pop(); //top=top.next;
        return topElement;
    }
    public boolean contains(int data){
        return search(data)!=-1;
    }
    public int size(){
        int count=0;
        Node temp=top;
        while(temp!=null){
            count++;
            temp=temp.next;
        }
        return count;
    }
}

```

```

private boolean isEmpty(){
    return top==null;
}
public void clear(){
    top=null;
}
public void printStack(){
    Node temp=top;
    System.out.print("[");
    while(temp!=null){
        System.out.print(temp.data+", ");
        temp=temp.next;
    }
    System.out.println(top==null ? "empty]":"\b\b]");
}
}
class Node{
    int data;
    Node next;
    Node(int data){
        this.data=data;
    }
}
class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.push(10);
        s1.push(20);
        s1.push(30);
        s1.push(40);
        s1.push(50);
        s1.printStack(); //[50,40,30,20,10]
        int topElement=s1.peek();
        System.out.println("Top Element : "+topElement);//50

        System.out.println();
        System.out.println("After calling peek()...");
        s1.printStack(); //[50,40,30,20,10]
        System.out.println();

        topElement=s1.poll();
        System.out.println("Top Element : "+topElement);//50

        System.out.println();
        System.out.println("After calling poll()...");
        s1.printStack(); //[40,30,20,10]
    }
}

```

```
class Stack{
    private Node top;

    public void push(int data){
        Node n1=new Node(data);
        n1.next=top;
        top=n1;
    }
    public void pop(){
        if(top!=null){
            top=top.next;
        }
    }
    public int search(int data){
        Node temp=top;
        int index=0;
        while(temp!=null){
            if(temp.data==data){
                return index;
            }
            index++;
            temp=temp.next;
        }
        return -1;
    }
    public int peek(){
        return isEmpty()? -1:top.data;
    }
    public int poll(){
        int topElement=isEmpty()? -1:top.data;
        pop(); //top=top.next;
        return topElement;
    }
    public boolean contains(int data){
        return search(data)!=-1;
    }
    public int size(){
        int count=0;
        Node temp=top;
        while(temp!=null){
            count++;
            temp=temp.next;
        }
        return count;
    }
}
```

```

        private boolean isEmpty(){
            return top==null;
        }
        public void clear(){
            top=null;
        }
        public void printStack(){
            Node temp=top;
            System.out.print("[");
            while(temp!=null){
                System.out.print(temp.data+", ");
                temp=temp.next;
            }
            System.out.println(top==null ? "empty]":"\b\b]");
        }
    }

    class Node{
        int data;
        Node next;
        Node(int data){
            this.data=data;
        }
    }

    class Demo{
        public static void main(String args[]){
            Stack s1=new Stack();
            s1.push(10);
            s1.push(20);
            s1.push(30);
            s1.push(40);
            s1.push(50);
            s1.printStack(); //[50,40,30,20,10]

            int[] ar=s1.toArray();

            System.out.println(Arrays.toString(ar));//[50,40,30,20,10]
        }
    }

```



```

        return count;
    }
    private boolean isEmpty(){
        return top==null;
    }
    public void clear(){
        top=null;
    }
    public void printStack(){
        Node temp=top;
        System.out.print("");
        while(temp!=null){
            System.out.print(temp.data+", ");
            temp=temp.next;
        }
        System.out.println(top==null ? "empty":"\b\b");
    }
    public int[] toArray(){
        int[] tempArray=new int[size()];
        int i=0;
        Node temp=top;
        while(temp!=null){
            tempArray[i++]=temp.data;
            temp=temp.next;
        }
        return tempArray;
    }
}

class Node{
    int data;
    Node next;
    Node(int data){
        this.data=data;
    }
}

class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.push(10);
        s1.push(20);
        s1.push(30);
        s1.push(40);
        s1.push(50);
        s1.printStack(); //[50,40,30,20,10]
        int[] ar=s1.toArray();
        System.out.println(Arrays.toString(ar));//[50,40,30,20,10]
    }
}

```

=====

```
import java.util.*;

class Stack{
    private Node top;

    public void push(int data){
        Node n1=new Node(data);
        n1.next=top;
        top=n1;
    }
    public void pop(){
        if(top!=null){
            top=top.next;
        }
    }
    public int search(int data){
        Node temp=top;
        int index=0;
        while(temp!=null){
            if(temp.data==data){
                return index;
            }
            index++;
            temp=temp.next;
        }
        return -1;
    }
    public int peek(){
        return isEmpty()? -1:top.data;
    }
    public int poll(){
        int topElement=isEmpty()? -1:top.data;
        pop(); //top=top.next;
        return topElement;
    }

    public boolean contains(int data){
        return search(data)!=-1;
    }
    public int size(){
        int count=0;
        Node temp=top;
        while(temp!=null){
            count++;
            temp=temp.next;
        }
    }
}
```

```

        return count;
    }
    private boolean isEmpty(){
        return top==null;
    }
    public void clear(){
        top=null;
    }
    public void printStack(){
        Node temp=top;
        System.out.print("[");
        while(temp!=null){
            System.out.print(temp.data+", ");
            temp=temp.next;
        }
        System.out.println(top==null ? "empty"]":"\b\b]");
    }
    public int[] toArray(){
        int[] tempArray=new int[size()];
        int i=0;
        Node temp=top;
        while(temp!=null){
            tempArray[i++]=temp.data;
            temp=temp.next;
        }
        return tempArray;
    }
}

class Node{
    private int data;
    private Node next;
    Node(int data){
        this.data=data;
    }
}

class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.push(10);
        s1.push(20);
        s1.push(30);
        s1.push(40);
        s1.push(50);
        s1.printStack(); //[50,40,30,20,10]
        int[] ar=s1.toArray();
        System.out.println(Arrays.toString(ar));//[50,40,30,20,10]
    }
}

```



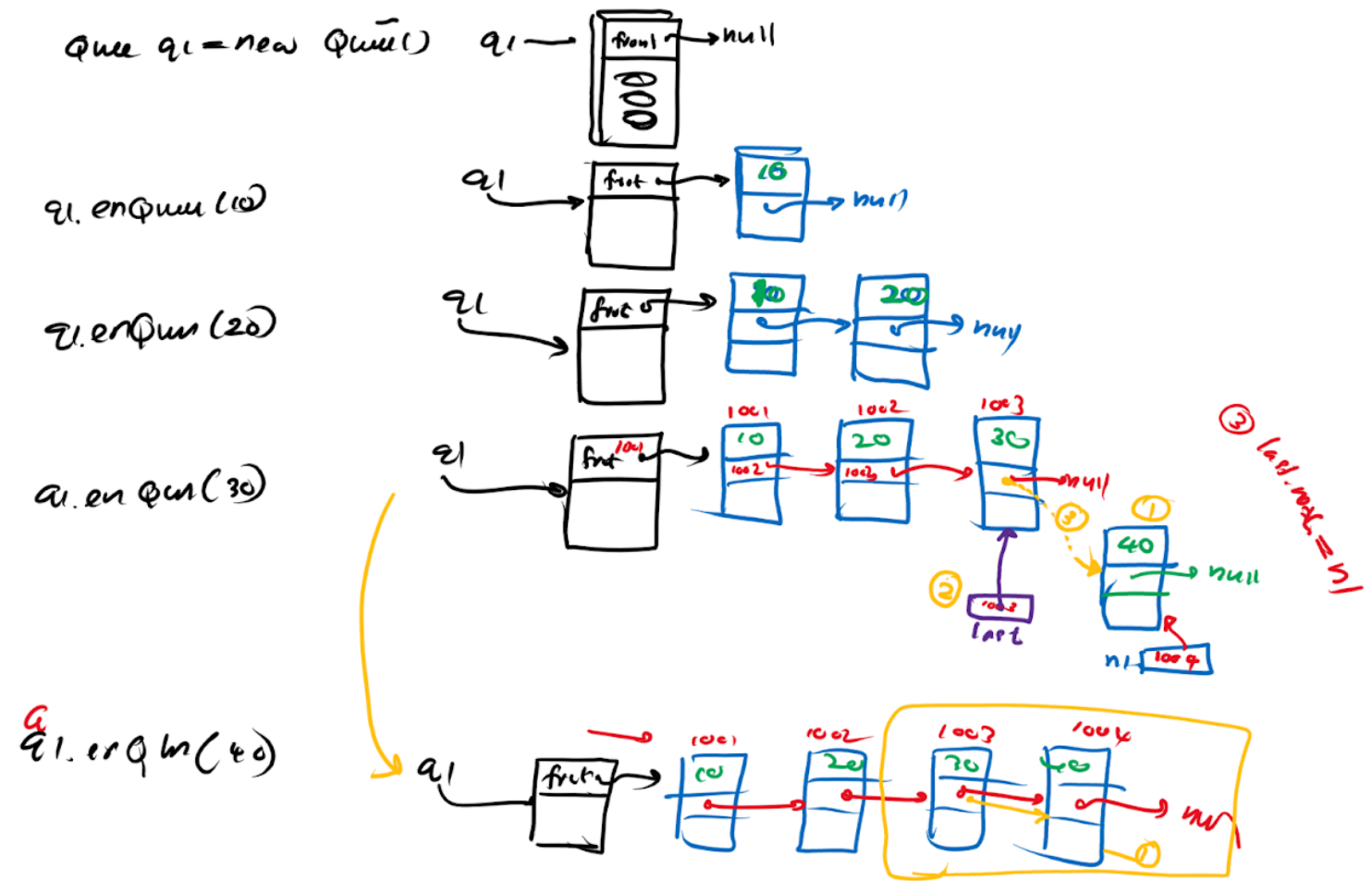
```

        return count;
    }
    private boolean isEmpty(){
        return top==null;
    }
    public void clear(){
        top=null;
    }
    public void printStack(){
        Node temp=top;
        System.out.print("");
        while(temp!=null){
            System.out.print(temp.data+", ");
            temp=temp.next;
        }
        System.out.println(top==null ? "empty":"\b\b");
    }
    public int[] toArray(){
        int[] tempArray=new int[size()];
        int i=0;
        Node temp=top;
        while(temp!=null){
            tempArray[i++]=temp.data;
            temp=temp.next;
        }
        return tempArray;
    }
    //-----Inner class-----
    class Node{
        private int data;
        private Node next;
        Node(int data){
            this.data=data;
        }
    }
}

class Demo{
    public static void main(String args[]){
        Stack s1=new Stack();
        s1.push(10);
        s1.push(20);
        s1.push(30);
        s1.push(40);
        s1.push(50);
        s1.printStack(); //[50,40,30,20,10]
        int[] ar=s1.toArray();
        System.out.println(Arrays.toString(ar));//[50,40,30,20,10]
    }
}

```

Class Queue



Q122

```
import java.util.*;

class Queue{
    private Node front;

    public void enqueue(int data){
        //1. Node n1=new Node(data);
        //2. find the lastNode
        //3. lastNode.next=n1;
    }

    public void dequeue(){
        if(front!=null){
            front=front.next;
        }
    }
}
```

```

public int search(int data){
    Node temp=front;
    int index=0;
    while(temp!=null){
        if(temp.data==data){
            return index;
        }
        index++;
        temp=temp.next;
    }
    return -1;
}

public int peek(){
    return isEmpty()? -1:front.data;
}

public int poll(){
    int frontElement=isEmpty()? -1:front.data;
    deQueue();
    return frontElement;
}

public boolean contains(int data){
    return search(data)!=-1;
}

public int size(){
    int count=0;
    Node temp=front;
    while(temp!=null){
        count++;
        temp=temp.next;
    }
    return count;
}

private boolean isEmpty(){
    return front==null;
}

public void clear(){
    front=null;
}

public void printQueue(){
    Node temp=front;
    System.out.print("[");
    while(temp!=null){
        System.out.print(temp.data+", ");
        temp=temp.next;
    }
    System.out.println(front==null ? "empty]":"\b\b]");
}

```

////////////////////////////////////

=====

```
import java.util.*;

class Queue{

    private Node front;

    public void enqueue(int data){
        Node n1=new Node(data);
        if(isEmpty()){
            front=n1;
        }else{
            Node lastNode=front;
```

```

        while(lastNode.next!=null){
            lastNode=lastNode.next;
        }
        lastNode.next=n1;
    }
}

public void deQueue(){
    if(front!=null){
        front=front.next;
    }
}

public int search(int data){
    Node temp=front;
    int index=0;
    while(temp!=null){
        if(temp.data==data){
            return index;
        }
        index++;
        temp=temp.next;
    }
    return -1;
}

public int peek(){
    return isEmpty()? -1:front.data;
}

public int poll(){
    int frontElement=isEmpty()? -1:front.data;
    deQueue();
    return frontElement;
}

public boolean contains(int data){
    return search(data)!=-1;
}

public int size(){
    int count=0;
    Node temp=front;
    while(temp!=null){
        count++;
        temp=temp.next;
    }
    return count;
}

private boolean isEmpty(){
    return front==null;
}

public void clear(){

```

```

        front=null;
    }
    public void printQueue(){
        Node temp=front;
        System.out.print("[");
        while(temp!=null){
            System.out.print(temp.data+", ");
            temp=temp.next;
        }
        System.out.println(front==null ? "empty]":"\b\b]");
    }
    public int[] toArray(){
        int[] tempArray=new int[size()];
        int i=0;
        Node temp=front;
        while(temp!=null){
            tempArray[i++]=temp.data;
            temp=temp.next;
        }
        return tempArray;
    }
    //-----Inner class-----
    class Node{
        private int data;
        private Node next;
        Node(int data){
            this.data=data;
        }
    }
}

class Demo{
    public static void main(String args[]){
        Queue q1=new Queue();
        q1.enqueue(10);
        q1.enqueue(20);
        q1.enqueue(30);
        q1.enqueue(40);
        q1.enqueue(50);
        q1.printQueue(); //[10,20,30,40,50]

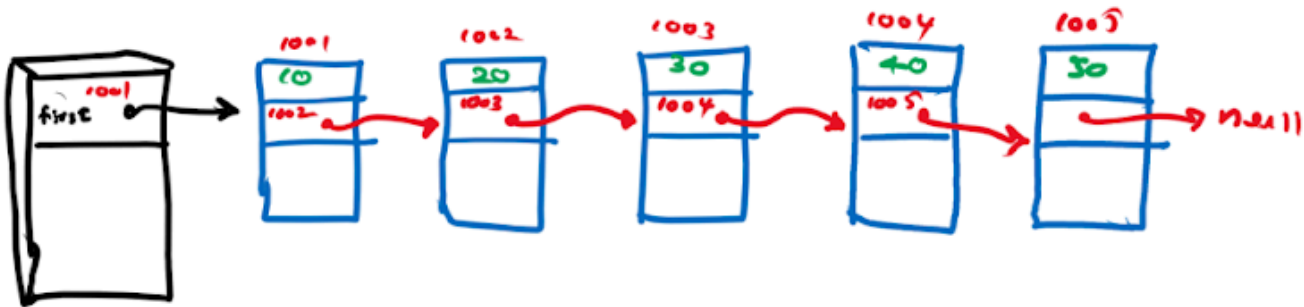
        q1.dequeue();
        q1.printQueue(); //[20,30,40,50]

    }
}

```

Class : List

List(indexing)



add(data) → enqueue.

add(index, data)

remove(index)

remove(start index, end index)

get(index) → data

set

Q124

```
import java.util.*;
```

```
class List{
```

```
    private Node first;
```

```
    public void add(int index, int data){
```

```
        //
```

```
        //
```

```
        //
```

```
        //
```

```
    }
```

```
    public void add(int data){
```

```
        Node n1=new Node(data);
```

```
        if(isEmpty()){
```

```
            first=n1;
```

```
        }else{
```

```
            Node lastNode=first;
```

```

        while(lastNode.next!=null){
            lastNode=lastNode.next;
        }
        lastNode.next=n1;
    }
}

public void remove(){ //remove the first element
    if(first!=null){
        first=first.next;
    }
}

public int search(int data){
    Node temp=first;
    int index=0;
    while(temp!=null){
        if(temp.data==data){
            return index;
        }
        index++;
        temp=temp.next;
    }
    return -1;
}

public boolean contains(int data){
    return search(data)!=-1;
}

public int size(){
    int count=0;
    Node temp=first;
    while(temp!=null){
        count++;
        temp=temp.next;
    }
    return count;
}

private boolean isEmpty(){
    return first==null;
}

public void clear(){
    first=null;
}

public void printList(){
    Node temp=first;
    System.out.print("[");
    while(temp!=null){
        System.out.print(temp.data+", ");
        temp=temp.next;
    }
}

```



```

        int count=0;
        Node temp=first;
        while(count<index-1){
            temp=temp.next;
            count++;
        }
        n1.next=temp.next;
        temp.next=n1;
    }
}

public void add(int data){
    Node n1=new Node(data);
    if(isEmpty()){
        first=n1;
    }else{
        Node lastNode=first;
        while(lastNode.next!=null){
            lastNode=lastNode.next;
        }
        lastNode.next=n1;
    }
}

public void remove(){ //remove the first element
    if(first!=null){
        first=first.next;
    }
}

public int search(int data){
    Node temp=first;
    int index=0;
    while(temp!=null){
        if(temp.data==data){
            return index;
        }
        index++;
        temp=temp.next;
    }
    return -1;
}

public boolean contains(int data){
    return search(data)!=-1;
}

public int size(){
    int count=0;
    Node temp=first;
    while(temp!=null){
        count++;
        temp=temp.next;
    }
}

```

```

        }
        return count;
    }
    private boolean isEmpty(){
        return first==null;
    }
    public void clear(){
        first=null;
    }
    public void printList(){
        Node temp=first;
        System.out.print("[");
        while(temp!=null){
            System.out.print(temp.data+", ");
            temp=temp.next;
        }
        System.out.println(first==null ? "empty]":"\b\b]");
    }
    public int[] toArray(){
        int[] tempArray=new int[size()];
        int i=0;
        Node temp=first;
        while(temp!=null){
            tempArray[i++]=temp.data;
            temp=temp.next;
        }
        return tempArray;
    }
    //-----Inner class-----
    class Node{
        private int data;
        private Node next;
        Node(int data){
            this.data=data;
        }
    }
}

class Demo{
    public static void main(String args[]){
        List list1=new List();
        list1.add(10);
        list1.add(20);
        list1.add(30);
        list1.add(40);
        list1.add(50);
        list1.printList(); //[10,20,30,40,50]
    }
}

```

```
list1.add(3,33);
list1.printList(); //[10,20,30,33,40,50]
}
```

Q126 Exercise (NullPointerException)

```
import java.util.*;

class List{
    private Node first;

    public void add(int index, int data){
        Node n1=new Node(data);

        if(index>=0 && index<=size()){
            int count=0;
            Node temp=first;
            while(count<index-1){
                temp=temp.next;
                count++;
            }
            n1.next=temp.next;
            temp.next=n1;
        }
    }

    public void add(int data){
        Node n1=new Node(data);
        if(isEmpty()){
            first=n1;
        }else{
            Node lastNode=first;
            while(lastNode.next!=null){
                lastNode=lastNode.next;
            }
            lastNode.next=n1;
        }
    }

    public void remove(){ //remove the first element
        if(first!=null){
            first=first.next;
        }
    }

    public int search(int data){
        Node temp=first;
        int index=0;
        while(temp!=null){
            if(temp.data==data){
```

```

        return index;
    }
    index++;
    temp=temp.next;
}
return -1;
}
public boolean contains(int data){
    return search(data)!=-1;
}
public int size(){
    int count=0;
    Node temp=first;
    while(temp!=null){
        count++;
        temp=temp.next;
    }
    return count;
}
private boolean isEmpty(){
    return first==null;
}
public void clear(){
    first=null;
}
public void printList(){
    Node temp=first;
    System.out.print("[");
    while(temp!=null){
        System.out.print(temp.data+", ");
        temp=temp.next;
    }
    System.out.println(first==null ? "empty]":"\b\b]");
}
public int[] toArray(){
    int[] tempArray=new int[size()];
    int i=0;
    Node temp=first;
    while(temp!=null){
        tempArray[i++]=temp.data;
        temp=temp.next;
    }
    return tempArray;
}
//-----Inner class-----
class Node{
    private int data;
    private Node next;

```



```

        }else{
            Node lastNode=first;
            while(lastNode.next!=null){
                lastNode=lastNode.next;
            }
            lastNode.next=n1;
        }
    }

    public void remove(){ //remove the first element
        if(first!=null){
            first=first.next;
        }
    }

    public int search(int data){
        Node temp=first;
        int index=0;
        while(temp!=null){
            if(temp.data==data){
                return index;
            }
            index++;
            temp=temp.next;
        }
        return -1;
    }

    public boolean contains(int data){
        return search(data)!=-1;
    }

    public int size(){
        int count=0;
        Node temp=first;
        while(temp!=null){
            count++;
            temp=temp.next;
        }
        return count;
    }

    private boolean isEmpty(){
        return first==null;
    }

    public void clear(){
        first=null;
    }

    public void printList(){
        Node temp=first;
        System.out.print("[");
        while(temp!=null){
            System.out.print(temp.data+", ");

```

```

        temp=temp.next;
    }
    System.out.println(first==null ? "empty":"\b\b");
}
public int[] toArray(){
    int[] tempArray=new int[size()];
    int i=0;
    Node temp=first;
    while(temp!=null){
        tempArray[i++]=temp.data;
        temp=temp.next;
    }
    return tempArray;
}
//-----Inner class-----
class Node{
    private int data;
    private Node next;
    Node(int data){
        this.data=data;
    }
}

}
class Demo{
    public static void main(String args[]){
        List list1=new List();
        list1.add(0,10);
        list1.add(0,20);
        list1.add(0,30);
        list1.add(0,40);
        list1.add(0,50);
        list1.printList(); //[50,40,30,20,10]

        list1.add(list1.size(),100);
        list1.add(list1.size(),200);
        list1.add(list1.size(),300);
        list1.add(list1.size(),400);
        list1.printList(); //[50,40,30,20,10,100,200,300,400]

        list1.add(5,11);
        list1.add(5,22);
        list1.add(5,33);
        list1.add(5,44);
        list1.add(5,55);
        list1.printList(); //[50,40,30,20,10,55,44,33,22,11,100,200,300,400]
    }
}

```

```
return index;
```

```

        }
        index++;
        temp=temp.next;
    }
    return -1;
}
public boolean contains(int data){
    return search(data)!=-1;
}
public int size(){
    int count=0;
    Node temp=first;
    while(temp!=null){
        count++;
        temp=temp.next;
    }
    return count;
}
private boolean isEmpty(){
    return first==null;
}
public void clear(){
    first=null;
}
public void printList(){
    Node temp=first;
    System.out.print("[");
    while(temp!=null){
        System.out.print(temp.data+", ");
        temp=temp.next;
    }
    System.out.println(first==null ? "empty]":"\b\b]");
}
public int[] toArray(){
    int[] tempArray=new int[size()];
    int i=0;
    Node temp=first;
    while(temp!=null){
        tempArray[i++]=temp.data;
        temp=temp.next;
    }
    return tempArray;
}
public int get(int index){

}
//-----Inner class-----
class Node{

```



```

        }
    }
}

public void add(int data){
    Node n1=new Node(data);
    if(isEmpty()){
        first=n1;
    }else{
        Node lastNode=first;
        while(lastNode.next!=null){
            lastNode=lastNode.next;
        }
        lastNode.next=n1;
    }
}

public void remove(){ //remove the first element
    if(first!=null){
        first=first.next;
    }
}

public int search(int data){
    Node temp=first;
    int index=0;
    while(temp!=null){
        if(temp.data==data){
            return index;
        }
        index++;
        temp=temp.next;
    }
    return -1;
}

public boolean contains(int data){
    return search(data)!=-1;
}

public int size(){
    int count=0;
    Node temp=first;
    while(temp!=null){
        count++;
        temp=temp.next;
    }
    return count;
}

private boolean isEmpty(){
    return first==null;
}

public void clear(){
    first=null;
}

public void printList(){

```

```

        Node temp=first;
        System.out.print("[");
        while(temp!=null){
            System.out.print(temp.data+", ");
            temp=temp.next;
        }
        System.out.println(first==null ? "empty]":"\b\b]");
    }
    public int[] toArray(){
        int[] tempArray=new int[size()];
        int i=0;
        Node temp=first;
        while(temp!=null){
            tempArray[i++]=temp.data;
            temp=temp.next;
        }
        return tempArray;
    }
    public int get(int index){
        if(index>=0 && index<size()){
            int count=0;
            Node temp=first;
            while(count<index){
                temp=temp.next;
                count++;
            }
            return temp.data;
        }
        return -1;
    }
    //-----Inner class-----
    class Node{
        private int data;
        private Node next;
        Node(int data){
            this.data=data;
        }
    }
}

class Demo{
    public static void main(String args[]){
        List list1=new List();
        list1.add(10);
        list1.add(20);
        list1.add(30);
        list1.add(40);
        list1.add(50);
        list1.printList(); //[10,20,30,40,50]

        int data=list1.get(3);
    }
}

```

```
data=list1.get(5);
System.out.println("Element of index 5 : "+data); //-1
    }
}
```

```
Q130 public void remove(index)
=====
```

```

}
public int search(int data){
    Node temp=first;
    int index=0;
    while(temp!=null){
        if(temp.data==data){
            return index;
        }
        index++;
        temp=temp.next;
    }
    return -1;
}
public boolean contains(int data){
    return search(data)!=-1;
}
public int size(){
    int count=0;
    Node temp=first;
    while(temp!=null){
        count++;
        temp=temp.next;
    }
    return count;
}
private boolean isEmpty(){
    return first==null;
}
public void clear(){
    first=null;
}
public void printList(){
    Node temp=first;
    System.out.print("[");
    while(temp!=null){
        System.out.print(temp.data+", ");
        temp=temp.next;
    }
    System.out.println(first==null ? "empty]":"\b\b]");
}
public int[] toArray(){
    int[] tempArray=new int[size()];
    int i=0;
    Node temp=first;
    while(temp!=null){
        tempArray[i++]=temp.data;
        temp=temp.next;
    }
}

```

```

        return tempArray;
    }
    public int get(int index){
        if(index>=0 && index<size()){
            int count=0;
            Node temp=first;
            while(count<index){
                temp=temp.next;
                count++;
            }
            return temp.data;
        }
        return -1;
    }
    public void remove(int index){

    }
    //-----Inner class-----
    class Node{
        private int data;
        private Node next;
        Node(int data){
            this.data=data;
        }
    }
}

class Demo{
    public static void main(String args[]){
        List list1=new List();
        list1.add(10);
        list1.add(20);
        list1.add(30);
        list1.add(40);
        list1.add(50);
        list1.printList(); //[10,20,30,40,50]

        list1.remove(3);
        list1.printList(); //[10,20,30,50]
    }
}

```



Q131

}